DOCUMENT DE RENDU

README — Concert Agent

Présentation du projet, installation, configuration et scénarios de démonstration.

CA

Concert Agent · Octobre 2026

Un assistant qui trouve de vrais concerts, explique d'où viennent les informations et demande la permission avant d'enregistrer un favori.

Concert Agent est une application **Python / Flask**. L'utilisateur écrit en français ; un **agent LLM** choisit les outils adaptés, Python les exécute, puis l'agent répond avec les résultats réels. Les concerts viennent de **Ticketmaster** ou, à défaut, d'un jeu de démonstration clairement signalé.

> **Démo en ligne :** https://concert-agent.onrender.com
>
> Le service gratuit peut mettre environ une minute à se réveiller. Sa base SQLite est remise à zéro à chaque redémarrage.

En 30 secondes

Question	Réponse
Quel problème ?	Un chatbot seul peut inventer une date, une salle ou un prix.
Quelle solution ?	Le LLM ne répond pas directement : il choisit des tools Python qui cherchent les concerts dans une vraie API.
Comment le vérifier ?	3 scénarios de démonstration, des traces pour chaque tool et 15 tests automatiques.
Pourquoi c'est sûr ?	Aucune écriture en base sans confirmation explicite de l'utilisateur.

Ce que tu peux faire

- Discuter en français pour chercher un concert.
- Chercher sur tous tes artistes suivis, avec un plafond clair de 40 artistes et 30 résultats.
- Filtrer par artiste, ville, région, date ou budget, y compris « cette année ».
- Connecter Spotify et importer tes artistes les plus écoutés.
- Ajouter un concert aux favoris après confirmation.
- Cliquer sur un favori pour voir sa fiche complète : date, heure, salle, ville, région, pays, prix, source et lien de billetterie.
- Consulter l'historique et les traces des tools.

Les 3 démonstrations

1. Chercher selon mes goûts

> « Trouve un concert de mes artistes préférés en Auvergne-Rhône-Alpes le mois prochain. »

L'agent consulte les préférences, interroge Ticketmaster, filtre les résultats et présente chaque concert avec sa date, son heure, son lieu, son prix et son lien de billetterie. Exemple de résultat réel : **Gims à la LDLC Arena de Décines**.

2. Savoir dire « je n'ai rien trouvé »

> « Je veux voir un concert à Paris demain, max 5 €. »

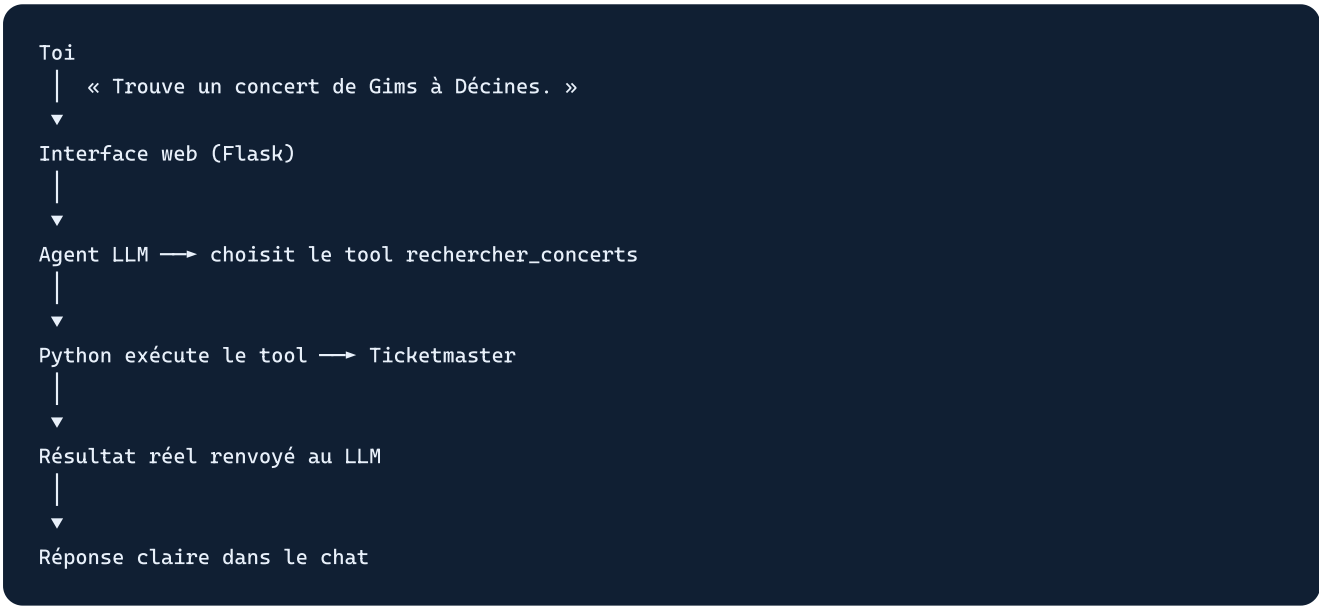
L'agent cherche réellement. S'il ne trouve rien, il le dit et n'invente ni date, ni salle, ni prix.

3. Demander avant d'enregistrer

> « Ajoute le concert de Gims à Décines à mes favoris. »

L'agent prépare l'action et affiche **« Confirmer ? »**. Le favori n'est écrit dans SQLite qu'après un clic sur **Oui**. Un clic sur **Non** annule l'action.

Comment ça marche



1. **Tu écris une demande** en français.

2. **Le LLM comprend l'intention** et choisit un tool : consulter les préférences, chercher des concerts, lister les favoris…
3. **Python exécute le tool.** Le LLM ne fabrique pas les concerts : il reçoit les données du tool.
4. **Le LLM rédige la réponse** à partir de ces données.
5. **Pour une écriture**, le code Python met l'action en attente et demande confirmation.
6. **Tout est tracé** : interaction, tool, arguments, résultat, statut et durée.

Les mots du projet

Mot	Définition simple
LLM	Modèle de langage qui comprend la demande et choisit quoi faire. Ici : Gemini.
Agent	Programme qui enchaîne plusieurs actions pour atteindre un objectif.
Tool	Fonction Python que l'agent peut appeler, par exemple `rechercher_concerts`.
SQLite	Petite base de données locale qui garde les préférences, favoris et traces.
Trace	Compte rendu d'un appel de tool : nom, arguments, résultat, durée.
Fallback	Solution de secours : mode règles si le LLM est indisponible, données de démo si Ticketmaster ne renvoie rien.
OAuth	Procédure officielle qui permet de connecter Spotify sans connaître ton mot de passe.

Pourquoi on peut lui faire confiance

- **Les concerts viennent d'un tool**, pas de l'imagination du modèle.
- **Les erreurs API sont visibles** : l'application distingue une panne, une absence de résultat et des données de démonstration.
- **Les données de démo sont étiquetées** `source=demo` et affichées avec un avertissement.
- **La confirmation est dans le code Python**, pas seulement dans une consigne donnée au LLM.
- **Les traces permettent de vérifier** ce qui a réellement été appelé.
- **Les secrets restent dans** `.env`, jamais dans Git ni dans le navigateur.

Démarrage rapide (Windows)

Option 1 — Utiliser la démo en ligne

Ouvrir https://concert-agent.onrender.com. Aucune installation nécessaire.

Option 2 — Lancer en local

1. Double-cliquer sur `run.bat`.
2. Ouvrir http://127.0.0.1:5000.
3. Pour arrêter le serveur : `Ctrl+C` dans la fenêtre noire.

Le script crée `.venv`, installe `requirements.txt` et lance Flask.

Démarrage manuel

```
py -3 -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements.txt
.venv\Scripts\python.exe app.py
```

Configuration

Copier `.env.example` vers `.env`, puis renseigner les clés utiles.

LLM (Gemini)

```
GEMINI_API_KEY=ta_cle_google_ai_studio
LLM_PROVIDER=gemini
LLM_MODEL=gemini-3.5-flash-lite
```

Clé gratuite : https://aistudio.google.com/apikey

Sans clé, l'application fonctionne en **mode règles** : même interface, mêmes tools, mêmes traces, mais les décisions sont prises par des règles Python au lieu du LLM.

Ticketmaster

```
TICKETMASTER_API_KEY=ta_cle_ticketmaster
DEMO_FALLBACK=1
MODE_DEMO=0
```

Avec une clé, l'API Discovery est interrogée en français (`locale=fr-fr`, `countryCode=FR`). L'application cherche d'abord l'artiste exact pour éviter les homonymes, puis récupère ses événements en France.

Sans clé, ou si aucun événement ne correspond, elle utilise les concerts de démonstration et l'indique dans la réponse. `MODE_DEMO=1` force ce mode.

Spotify

```
SPOTIFY_CLIENT_ID=ton_client_id
SPOTIFY_CLIENT_SECRET=ton_client_secret
REDIRECT_URI=http://127.0.0.1:5000/callback
```

1. Ouvrir https://developer.spotify.com/dashboard.
2. Dans **Redirect URIs**, ajouter exactement `http://127.0.0.1:5000/callback`.
3. Dans **User Management**, ajouter les comptes autorisés (5 maximum en mode développement).
4. Redémarrer `run.bat`, puis cliquer sur **Se connecter avec Spotify**.

Spotify n'accepte plus l'alias `localhost` : utiliser l'adresse loopback explicite `127.0.0.1`. L'import ajoute jusqu'à 30 artistes (top artistes + écoutes récentes), sans doublons.

Tests

```
.venv\Scripts\python.exe -m unittest discover -s tests -v
```

Les tests couvrent les 3 scénarios, la confirmation, le refus, l'historique, les préférences, les tools et l'intégration Spotify sans appel réseau.

Résultat attendu : `Ran 15 tests ... OK`.

Structure du projet

Fichier / dossier	Rôle
`app.py`	Serveur Flask et routes HTTP.
`db.py`	Base SQLite, schéma, données de départ et historique.
`agent/loop.py`	Boucle de l'agent et confirmation des écritures.
`agent/llm.py`	Point d'appel unique du modèle de langage.
`agent/tools.py`	Les 6 tools et les appels Ticketmaster.
`agent/spotify.py`	Connexion OAuth et import des artistes Spotify.
`agent/regles.py`	Mode de secours sans LLM.
`agent/villes.py`	Villes, régions, accents et normalisation.
`templates/index.html`	Interface web.
`static/style.css`	Thème visuel.
`static/app.js`	Logique du navigateur et fiche détaillée des favoris.
`tests/test_agent.py`	Tests automatiques.
`run.bat`	Lancement Windows.
`render.yaml`	Déploiement Render.
`Procfile`	Démarrage Gunicorn.

Déploiement Render

Le dossier à publier est `concert-agent-code`, pas l'ancien projet Node `concert-alert`.

1. Créer un dépôt GitHub `concert-agent`.
2. Envoyer **le contenu** de `concert-agent-code` à la racine du dépôt. Ne jamais envoyer `.env`, `data/` ni `.venv/`.
3. Sur Render : **New > Blueprint**, connecter GitHub, sélectionner le dépôt, puis **Apply**.
4. Renseigner `GEMINI_API_KEY`, `TICKETMASTER_API_KEY`, `SPOTIFY_CLIENT_ID` et `SPOTIFY_CLIENT_SECRET`.
5. Après le déploiement, mettre `REDIRECT_URI=https://<ton-service>.onrender.com/callback` dans Render **et** dans Spotify Dashboard.
6. Attendre le statut **Live**, puis ouvrir l'URL publique.

Le plan gratuit est éphémère et se met en veille : la base SQLite est réinitialisée et le premier chargement peut être lent. Pour conserver les données, utiliser un disque persistant payant ou une base Postgres.

Limites connues

- La couverture de Ticketmaster en France est partielle : certains artistes n'ont aucun événement dans l'API.
- Ticketmaster ne fournit pas toujours les prix : l'application affiche alors `Prix : non communiqué`.
- Les concerts `source=demo` ne sont **pas** de vrais événements.
- Spotify en mode développement est limité à 5 comptes.
- Le plan Render gratuit ne conserve pas les favoris après un redémarrage.
- **Aucune notification automatique** n'est implémentée aujourd'hui. Les notifications, le partage de favoris et Bandsintown sont des perspectives, pas des fonctionnalités livrées.

FAQ

Est-ce que les concerts se mettent à jour souvent ?

Ticketmaster est interrogé **à chaque recherche**. Il n'y a pas de tâche de fond toutes les cinq minutes. Un favori est un instantané des informations au moment de l'ajout.

Pourquoi aucun concert n'apparaît pour mon artiste ?

Ticketmaster ne couvre pas tous les artistes ni toutes les dates en France. L'application le signale et peut basculer sur des données de démonstration.

Pourquoi je ne vois pas tous mes artistes suivis ?

Une recherche sans artiste précis interroge **tous les artistes suivis**, dans la limite de **40**. Les résultats sont ensuite plafonnés à **30 concerts** pour garder une réponse lisible. Si la liste ou le nombre de concerts dépasse ce plafond, l'agent l'indique dans sa réponse. Pour cibler un artiste absent de l'affichage, demande-le explicitement : « les concerts de Ninho cette année ».

Est-ce que « cette année » est compris ?

Oui. La période correspond au **1er janvier – 31 décembre de l'année en cours**. Les concerts passés ne sont pas proposés par Ticketmaster ; la recherche porte donc sur les dates à venir de l'année.

Puis-je utiliser l'application sans Spotify ?

Oui. Spotify sert seulement à importer automatiquement tes artistes. La recherche, les favoris et les liens de billetterie fonctionnent sans connexion Spotify.

Est-ce que le LLM peut inventer un concert ?

Le prompt lui interdit, mais surtout les concerts viennent des tools. `ajouter_favori` vérifie aussi que le concert provient de la dernière recherche. En cas de panne du LLM, le mode règles prend le relais.

Où sont stockées mes données ?

Dans SQLite, sur le serveur. En local : `data/concerts.db`. En ligne : la base Render, effacée à chaque redémarrage sur le plan gratuit.

Pour le jury : où regarder

Exigence	Preuve dans le projet
Application Python / Flask	`app.py`, `render.yaml`, `requirements.txt`
Interface Flask	`templates/index.html`, `static/`
LLM distant	`agent/llm.py`, badge LLM, `/api/etat`
LLM qui choisit les tools	`agent/loop.py`, panneau « Traces »
SQLite lue et écrite	`db.py`, tools `ajouter_favori` et `ajouter_artiste_suivi`
Confirmation avant écriture	`pending_actions`, boutons Oui/Non, tests
Historique et traces	tables `interactions` et `tool_traces`, `/api/historique`
3 scénarios	`DEMOS.md` / `DEMOS.pdf` et `tests/test_agent.py`
4 livrables	`README`, `DOCUMENTATION`, `DEMOS`, `PRESENTATION` + PDFs

Les livrables sont disponibles en Markdown et en PDF dans le dossier `pdf/`.